
HMI Clasificador de Paquetes

Informe Técnico – Manual de Usuario

Versión 1.1

Proyecto: HMI_Clasificador
Institución: Universidad Nacional de Entre Ríos (UNER)
Materia: Microcontroladores
Plataforma: Qt 6 / C++ – Windows
Repositorio: github.com/XradX314/HMI_Clasificador
Fecha: Mayo 2026

Índice general

I	Manual de Usuario	2
1.	Introducción	3
1.1.	Descripción general	3
1.2.	Requisitos del sistema	3
2.	Instalación y arranque	4
2.1.	Compilación desde código fuente	4
2.2.	Conexión del hardware	4
3.	Interfaz gráfica	5
3.1.	Barra de herramientas	5
3.2.	Panel Monitor	5
3.2.1.	Heartbeat / Conexión	5
3.2.2.	Panel de control	6
3.2.3.	Panel de salidas	6
3.2.4.	Panel de sensores y brazos	6
3.2.5.	Contadores	6
3.2.6.	Última caja	6
4.	Diálogos de configuración	7
4.1.	Calibración (CMD 0x63)	7
4.2.	Ancho de caja (CMD 0x62)	7
4.3.	Configuración avanzada (CMDs 0x64–0x67)	7
II	Documentación Técnica	8
5.	Arquitectura del sistema	9
5.1.	Vista general	9
5.2.	Diagrama de clases simplificado	9
5.3.	Flujo de datos	9
6.	Protocolo UNER	11
6.1.	Estructura de trama	11
6.2.	Tabla de comandos	11
6.3.	Verificación de checksum	12
6.4.	Máquina de estados del decodificador	12

7. API de clases	13
7.1. UnerProtocol	13
7.1.1. Encoder (métodos estáticos)	13
7.1.2. Decoder	13
7.2. SerialManager	14
7.2.1. Métodos públicos	14
7.2.2. Señales	14
7.3. Widgets	15
8. Implementación del protocolo – detalles	16
8.1. Corrección: máscara de bits en CMD_BRAZO (0x52)	16
8.2. CMD_TRIGGER (0x61) – trama sin payload	16
8.3. CMD_BLIND_DIST (0x60) – envío parcial	16
9. Estructura del repositorio	17
Referencias	18

Parte I

Manual de Usuario

Capítulo 1

Introducción

1.1. Descripción general

El **HMI Clasificador de Paquetes** es una aplicación de escritorio desarrollada en Qt 6 / C++ que permite controlar y monitorear un sistema automático de clasificación de cajas por cinta transportadora.

La comunicación con el microcontrolador (MCU) se realiza a través de un puerto serie (UART) utilizando el protocolo binario propio **UNER**. La aplicación ofrece:

- Control de arranque, parada y reinicio del clasificador.
- Configuración de las salidas de clasificación (tipo de caja asignado a cada rama de la cinta).
- Modo ciego: clasificación basada en distancias configurables en lugar de medición activa.
- Calibración de alturas de referencia, tiempos de brazos y parámetros de hardware.
- Visualización en tiempo real de sensores IR, estado de los brazos actuadores y contadores de cajas.
- Indicador de heartbeat (LED animado) con monitoreo de la conexión.
- Temas visual claro y oscuro.

1.2. Requisitos del sistema

Componente	Requisito mínimo
Sistema operativo	Windows 10 / 11 (64-bit)
Qt Runtime	Qt 6.x (incluido en el instalador)
Puerto USB	Adaptador USB-UART compatible con el MCU
MCU firmware	Compatible con protocolo UNER v1.1

Capítulo 2

Instalación y arranque

2.1. Compilación desde código fuente

1. Instalar Qt 6 con soporte para QSerialPort y MinGW.
2. Abrir `HMI_Clasificador.pro` en Qt Creator.
3. Seleccionar el kit de compilación MinGW 64-bit.
4. Presionar **Ejecutar** (Ctrl+R).

O desde línea de comandos (directorio del proyecto):

Listing 2.1: Compilación por consola

```
1 qmake HMI_Clasificador.pro -spec win32-g++  
2 mingw32-make -j4
```

2.2. Conexión del hardware

1. Conectar el MCU al PC mediante el adaptador USB-UART.
2. Verificar que el puerto COM aparece en el Administrador de dispositivos.
3. Abrir la aplicación `HMI_Clasificador.exe`.

Capítulo 3

Interfaz gráfica

3.1. Barra de herramientas

La barra superior contiene los controles de conexión serie:

- **Puerto COM:** selector del puerto serie disponible.
- **Baudios:** velocidad de transmisión (115200 recomendado).
- **Conectar / Desconectar:** establece o cierra la conexión.
- **Actualizar:** refresca la lista de puertos detectados.
- ◦ **Calibración:** abre el diálogo de calibración (CMD 0x63).
- **Ancho de caja:** abre el diálogo de ancho de referencia (CMD 0x62).
- **Config. avanzada:** abre el diálogo de parámetros de hardware (CMDs 0x64–0x67).
- **Modo oscuro:** alterna el tema visual.

3.2. Panel Monitor

3.2.1. Heartbeat / Conexión

Muestra el LED animado **LedAliveWidget** con los estados:

- **Verde:** heartbeat recibido (online).
- **Gris:** sin actividad (offline).
- **Naranja:** timeout (> 7 s sin heartbeat).

3.2.2. Panel de control

Control	CMD	Descripción
Start	0x50	Inicia la cinta con los tipos de caja configurados
Stop	0x51	Detiene la cinta
Reset	0x53	Reinicia el MCU y los contadores
Velocidad	0x54	Índice de velocidad (1–10; enviado al cambiar el SpinBox)
Modo Ciego	0x60	Activa/desactiva modo ciego + envía distancias S0→Sal0/1/2
Trigger	0x61	Dispara una medición manual inmediata

3.2.3. Panel de salidas

Tres *ComboBox* permiten asignar a cada salida el tipo de caja (Ninguna / Pequeña / Mediana / Grande). El botón **Aplicar** envía el CMD 0x50 con la configuración actual.

3.2.4. Panel de sensores y brazos

Muestra en tiempo real el estado de los 4 sensores IR y los 3 brazos actuadores. Los indicadores cambian de color al recibir los CMDs 0x5E (IR) y 0x52 (Brazo).

3.2.5. Contadores

Muestra la cantidad de cajas entradas, salidas y en tránsito durante la sesión actual.

3.2.6. Última caja

Muestra la altura medida (cm), el tipo inferido y el timestamp de la última caja detectada (CMD 0x5F).

Capítulo 4

Diálogos de configuración

4.1. Calibración (CMD 0x63)

Permite ajustar las alturas de referencia del sensor HC-SR04:

- **Distancia piso:** distancia del sensor al piso sin caja (`calibracion[0]`).
- **Caja pequeña / mediana / grande:** alturas de cada tipo (`calibracion[1..3]`).
- **Tolerancia:** margen de error en cm.
- **Extensión / Retracción:** tiempos de actuación de los brazos en ticks (1 tick = 2 ms).

Presionar **Enviar 0x63** transmite los 7 bytes al MCU.

4.2. Ancho de caja (CMD 0x62)

Permite enviar el ancho de la caja de referencia en cm. El MCU utiliza este valor para calcular internamente la velocidad de la cinta.

4.3. Configuración avanzada (CMDs 0x64–0x67)

Diálogo desplazable con cuatro secciones independientes:

- **HC-SR04 (0x64):** pulso de trigger (μ s) y timeout de eco (μ s).
- **Servo SG90 (0x65):** período, pulso mínimo, máximo y neutro (μ s).
- **Debounce IR (0x66):** número de ticks de debounce.
- **Timers / Cajas (0x67):** `retractTimer`, ángulos SG90, `hcsrTimer`, `aliveTimer` y `configCajas[0..2]`.

Cada sección tiene su propio botón **Enviar** para transmitir únicamente los parámetros de esa sección.

Parte II

Documentación Técnica

Capítulo 5

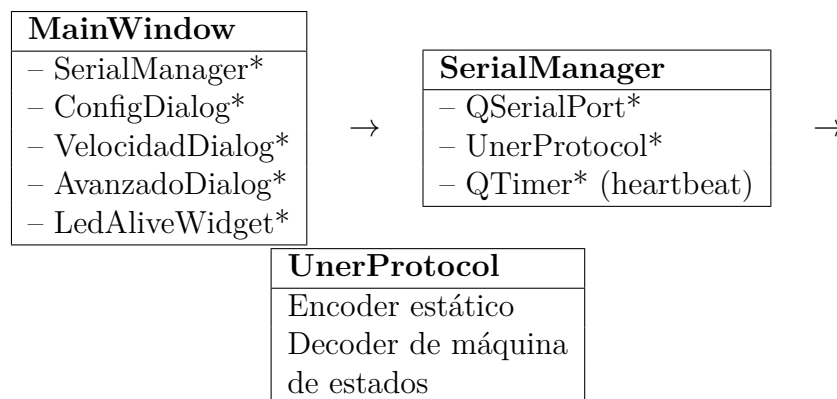
Arquitectura del sistema

5.1. Vista general

El sistema se organiza en tres capas:

1. **Capa de protocolo** (comunicacion/): codificación/decodificación de tramas UNER.
2. **Capa de comunicación** (comunicacion/): gestión del puerto serie y despacho de eventos.
3. **Capa de presentación** (MainWindow.cpp, widgets/): interfaz gráfica y lógica de usuario.

5.2. Diagrama de clases simplificado



5.3. Flujo de datos

1. El MCU envía bytes por UART.
2. `QSerialPort` emite `readyRead` → `SerialManager::onDataReady`.
3. Los bytes se pasan a `UnerProtocol::feed()` (máquina de estados).
4. Al completarse una trama válida, `UnerProtocol` emite `frameReceived`.

5. **SerialManager::dispatchFrame()** interpreta el comando y emite señales específicas (`cajaMedida`, `sensorIrActualizado`, `brazoActuado`, `aliveReceived`).
6. **MainWindow** recibe las señales y actualiza la UI.

Capítulo 6

Protocolo UNER

6.1. Estructura de trama

Toda trama sigue el formato:

U	N	E	R	len	':'
cmd	payload[0]	...	payload[n-1]	XOR	

- **Header:** 4 bytes fijos 'U', 'N', 'E', 'R'.
- **len:** 1 + n + 1 (cmd + payload + checksum).
- **':'** (0x3A): token separador.
- **cmd:** byte de comando.
- **payload:** 0 a n bytes de datos.
- **XOR:** XOR de todos los bytes anteriores de la trama.

6.2. Tabla de comandos

CMD	Nombre	Dirección	Payload
0xF0	CMD_ALIVE	MCU→HMI / HMI→MCU	[0x06] (ACK bidireccional)
0x50	CMD_START	HMI→MCU	[0x00, tipoSal0, tipoSal1, tipoSal2]
0x51	CMD_STOP	HMI→MCU	[0x06]
0x52	CMD_BRAZO	MCU→HMI	[máscara_servo, estado]
0x53	CMD_RESET	HMI→MCU	[0x06]
0x54	CMD_VELOCIDAD	HMI→MCU	[velIdx × 10] (0–100)
0x5E	CMD_IR_STATE	MCU→HMI	pares [outNum, IRState]
0x5F	CMD_CAJA_DETECT	MCU→HMI	[alturaCm]
0x60	CMD_BLIND_DIST	HMI→MCU	[modo_ciego, dist_s0[0], [1], [2] (1–4 bytes)
0x61	CMD_TRIGGER	HMI→MCU	(sin payload)

CMD	Nombre	Dirección	Payload
0x62	CMD_ANCHO_CAJA	HMI→MCU	[anchoCm]
0x63	CMD_CALIBRACION	HMI→MCU	[cal0, cal1, cal2, cal3, tol, ext, ret]
0x64	CMD_HCSR04_CFG	HMI→MCU	[trig_pulse_us, timeout_hi, timeout_lo]
0x65	CMD_SG90_CFG	HMI→MCU	[period_hi, lo, min_hi, lo, max_hi, lo, neutral_hi, lo]
0x66	CMD_IR_DEBOUNCE	HMI→MCU	[debounce_ticks]
0x67	CMD_TIMERS_CFG	HMI→MCU	[retract, angleDet, angleRep, hcsr, alive, cajas0, cajas1, cajas2]

6.3. Verificación de checksum

El checksum es el XOR acumulado de todos los bytes de la trama excepto el propio byte de checksum. El receptor verifica haciendo XOR de todos los bytes incluyendo el checksum: si el resultado es `0x00`, la trama es válida.

6.4. Máquina de estados del decodificador

El decodificador implementado en `UnerProtocol::feed()` procesa los bytes de entrada uno a uno siguiendo los estados:

WaitU → WaitN → WaitE → WaitR → ReadLen → WaitColon → ReadPayload

Al completarse `ReadPayload` con checksum correcto, se llama a `processPayload()` que emite la señal `frameReceived`.

Capítulo 7

API de clases

7.1. UnerProtocol

Archivo: comunicacion/UnerProtocol.h / .cpp

7.1.1. Encoder (métodos estáticos)

Método	Descripción
encode(cmd, payload)	Framer genérico; genera la trama completa.
ackAlive()	Genera ACK de heartbeat (0xF0).
cmdStart(s0,s1,s2)	Genera trama CMD_START (0x50).
cmdStop()	Genera trama CMD_STOP (0x51).
cmdReset()	Genera trama CMD_RESET (0x53).
cmdVelocidad(velIdx)	Genera trama CMD_VELOCIDAD (0x54).
cmdBlindDist(cfg,n)	Genera trama CMD_BLIND_DIST (0x60).
cmdTrigger()	Genera trama CMD_TRIGGER (0x61).
cmdAnchoCaja(cm)	Genera trama CMD_ANCHO_CAJA (0x62).
cmdCalibracion(cfg)	Genera trama CMD_CALIBRACION (0x63).
cmdHcsr04Cfg(cfg)	Genera trama CMD_HCSR04_CFG (0x64).
cmdSg90Cfg(cfg)	Genera trama CMD_SG90_CFG (0x65).
cmdIrDebounce(d)	Genera trama CMD_IR_DEBOUNCE (0x66).
cmdTimersCfg(cfg)	Genera trama CMD_TIMERS_CFG (0x67).

7.1.2. Decoder

Método / Señal	Descripción
feed(data)	Alimenta bytes al decodificador.
reset()	Reinicia la máquina de estados.
frameReceived(f)	Señal emitida al recibir una trama válida.
checksumError(raw)	Señal emitida ante error de checksum.

7.2. SerialManager

Archivo: comunicacion/SerialManager.h / .cpp

7.2.1. Métodos públicos

Método	Descripción
<code>open(port, baud)</code>	Abre el puerto serie e inicia el heartbeat.
<code>close()</code>	Cierra el puerto y detiene los timers.
<code>isOpen()</code>	Devuelve si el puerto está abierto.
<code>currentPort()</code>	Devuelve el nombre del puerto activo.
<code>availablePorts()</code>	Lista los puertos COM disponibles.
<code>sendStart(s0,s1,s2)</code>	Envía CMD_START.
<code>sendStop()</code>	Envía CMD_STOP.
<code>sendReset()</code>	Envía CMD_RESET.
<code>sendVelocidad(v)</code>	Envía CMD_VELOCIDAD.
<code>sendBlindDist(cfg,n)</code>	Envía CMD_BLIND_DIST.
<code>sendTrigger()</code>	Envía CMD_TRIGGER.
<code>sendAnchoCaja(cm)</code>	Envía CMD_ANCHO_CAJA.
<code>sendCalibracion(cfg)</code>	Envía CMD_CALIBRACION.
<code>sendHcsr04Cfg(cfg)</code>	Envía CMD_HCSR04_CFG.
<code>sendSg90Cfg(cfg)</code>	Envía CMD_SG90_CFG.
<code>sendIrDebounce(d)</code>	Envía CMD_IR_DEBOUNCE.
<code>sendTimersCfg(cfg)</code>	Envía CMD_TIMERS_CFG.

7.2.2. Señales

Señal	Descripción
<code>connected(port)</code>	Puerto abierto exitosamente.
<code>disconnected()</code>	Puerto cerrado.
<code>connectionLost()</code>	Timeout de heartbeat.
<code>errorOccurred(msg)</code>	Error en el puerto serie.
<code>aliveReceived()</code>	Heartbeat 0xF0 recibido.
<code>cajaMedida(cm)</code>	Caja medida (0x5F).
<code>sensorIrActualizado(num,act)</code>	Estado sensor IR (0x5E).
<code>brazoActuado(idx)</code>	Brazo actuado (0x52).
<code>frameReady(f)</code>	Cualquier trama decodificada.

7.3. Widgets

Clase	Responsabilidad
LedAliveWidget	LED circular animado de heartbeat con estadísticas.
ConfigDialog	Diálogo de calibración del clasificador (CMD 0x63).
VelocidadDialog	Diálogo de ancho de caja de referencia (CMD 0x62).
AvanzadoDialog	Diálogo de configuración avanzada de hardware (CMDs 0x64–0x67).

Capítulo 8

Implementación del protocolo – detalles

8.1. Corrección: máscara de bits en CMD_BRAZO (0x52)

El MCU envía en el byte `payload[0]` una *máscara de bits* donde el bit i indica que el servo i fue actuado:

Listing 8.1: Despacho de CMD_BRAZO en SerialManager

```
1 case Uner::CMD_BRAZO:
2     if (frame.payload.size() >= 2) {
3         const uint8_t mask = static_cast<uint8_t>(frame.payload
4             .at(0));
5         for (uint8_t i = 0; i < 3; i++) {
6             if (mask & (1 << i))
7                 emit brazoActuado(i);
8         }
9         break;
```

La versión anterior trataba `payload[0]` como un índice directo, lo que causaba que el servo 2 (máscara = 4) se ignorara por superar el límite.

8.2. CMD_TRIGGER (0x61) – trama sin payload

El codificador llama a `encode(CMD_TRIGGER)` sin payload. Esto genera `len = 2` (1 byte de cmd + 1 byte de checksum), que el MCU interpreta correctamente como un comando sin datos.

8.3. CMD_BLIND_DIST (0x60) – envío parcial

El parámetro `numBytes` (1–4) permite enviar solo el byte `modo_ciego` o también incluir las distancias `dist_s0[0..2]`, facilitando la compatibilidad con versiones de firmware que no soportan las distancias.

Capítulo 9

Estructura del repositorio

HMI_Clasificador/	
main.cpp	Punto de entrada
MainWindow.h / .cpp	Ventana principal
HMI_Clasificador.pro	Archivo de proyecto qmake
CLAUDE.md	Guía de arquitectura para Claude Code
Doxyfile	Configuración de Doxygen
comunicacion/	
UnerProtocol.h / .cpp	Protocolo UNER (encoder + decoder)
SerialManager.h / .cpp	Gestor de puerto serie
widgets/	
LedAliveWidget.h / .cpp	LED de heartbeat
ConfigDialog.h / .cpp	Diálogo calibración (0x63)
VelocidadDialog.h / .cpp	Diálogo ancho de caja (0x62)
AvanzadoDialog.h / .cpp	Diálogo config. avanzada (0x64-0x67)
docs/	
doxygen/	Documentación generada por Doxygen
informe/	Este informe técnico
Actividad-N-4-.../	Código fuente del firmware MCU (AVR C)

Referencias

- Qt Documentation – doc.qt.io/qt-6/
- Qt Serial Port Module – [QtSerialPort](#)
- Repositorio del proyecto – github.com/XradX314/HMI_Clasificador
- Documentación Doxygen – doxygen.nl